Conv_Simple — Complete

Bench-rig walkthrough, end-to-end: RS-485 wiring → Prog3-Modbus polling sequencer → motor_starter UDFB → v4.1.9 program cleanup

---

Audience: First-time CCW developer with a Micro 820 + GS10 bench rig on the desk | **PLC:** Allen-Bradley Micro 820 (2080-LC20-20QBB)
Drive: AutomationDirect GS10 DURApulse, Modbus RTU slave node 1
Project: `MIRA_v5` | **IDE:** Connected Components Workbench (CCW) v22+
Source-of-truth program: `plc/Micro820_v4.1.8_Program.st` (will become `v4.1.9` after applying Part 3)
Primary references: *GS10 DURApulse User Manual* + `plc/GS10_Integration_Guide.md` + *Micro 820 ref manual*
This doc supersedes: `Conv_Simple_Prog3_Modbus_Polling.pdf`, `Conv_Simple_UDFB_Intro.pdf`, `plc/LD_BUILD_GUIDE_v5.md` (v5.1)
You will understand after reading: exactly which wires go where; why the Modbus sequencer is a state machine driven by `vfd_poll_step`; what a UDFB is and why one instance of `motor_starter` beats five copy-pasted rungs; which lines of the deployed `.st` still need surgery and what each fix looks like; and which numbers must be bench-verified before relying on production.

📖 Reading map (three parts, each self-contained but listed in execution order)

1. **Part 1 — Modbus polling sequencer (Prog3-Modbus)**. Without this the drive doesn't talk. Sections 1–9. *~12 pages.*
2. **Part 2 — motor_starter UDFB**. Wraps the run/stop/direction logic into one reusable block. Assumes Part 1's wiring works. Sections 10–19. *~14 pages.*
3. **Part 3 — Apply to v4.1.9**. A line-by-line checklist that turns the deployed `Micro820_v4.1.8_Program.st` into v4.1.9 by applying every fix flagged in Parts 1 and 2. Section 20. *~4 pages.*

ⓘ Unified one-page cheat sheet — every value that absolutely must be right

Bench wiring + serial parameters

What	Value	Source
PLC serial channel (embedded RS-485)	`2` **(NOT 0 — v5.1 LD guide is wrong; v4.1.8 .st is wrong)**	Micro 820 ref manual; `GS10_Integration_Guide.md` §5
Drive RJ45 pin for D+ (SG+)	**Pin 5** (green/white per T568B)	GS10 manual p.10
Drive RJ45 pin for D- (SG-)	**Pin 4** (blue)	GS10 manual p.10
Drive RJ45 pin for signal ground	**Pin 3** (SGND)	GS10 manual p.10
VFD slave address (Modbus Node, P09.00)	`1`	GS10 manual p.6

What	Value	Source
Baud / format	`9600 8N2 RTU` (P09.01=96, P09.04=13)	GS10 manual p.6

GS10 register map

What	Register	Notes
Control Command (FC06 write)	`0x2000` (dec `8192`)	Bit-field integer
Frequency Setpoint (FC06 write)	`0x2001` (dec `8193`)	Value = Hz × 10 (300 = 30.0 Hz)
Fault Reset (FC06 write)	`0x2002` (dec `8194`)	Write `2` to clear non-latched faults
Operation Status WORD (FC03 read)	`0x2101` (dec `8449`)	NOT in v4.1.8 — to add in v4.1.9
Monitor block (FC03 read)	Start `0x2103` (dec `8451`), `ElementCnt = 4`	Covers 0x2103–0x2106 (freq, current, DC bus, V_out). **Reading 6 lands in undocumented 0x2108 — do NOT.**

Command word values (bench-verify before production)

Command	Decimal	Hex	Note
STOP	`1`	`0x0001`	Confirmed
FWD + RUN	`18`	`0x0012`	Deployed value (.st line 127, 146). Bit-math across sources contradicts (10/18/34). **Bench-verify.**
REV + RUN	`20`	`0x0014`	Deployed value (.st line 129, 148). Same contradiction. See Section 18 open question.

motor_starter UDFB interface

Pin	Type	Wired to (this conveyor)
`Enable`	BOOL in	`enable_motor_ctrl` (new global, default TRUE)
`StartPB`	BOOL in	`_IO_EM_DI_04` (PBRun)
`DirFwdSw`	BOOL in	`dir_fwd` (decoded in Prog2 §2)
`DirRevSw`	BOOL in	`dir_rev` (decoded in Prog2 §2)
`EStopOK`	BOOL in	`e_stop_ok` (new global, computed in Prog2 §1)
`VFDCmdWord`	INT out	`vfd_cmd_word` (sole writer after v4.1.9)
`RunLamp`	BOOL out	`_IO_EM_DO_00` (LightGreen)

> **⚠ Carry-forward bug log — what's wrong in the prior docs AND in `Micro820_v4.1.8_Program.st` that this complete walkthrough corrects**
>
> **From the v5.1 LD build guide (`plc/LD_BUILD_GUIDE_v5.md`):**

1. **Channel = 0** in every config table. Correct = `2`. The Micro 820's embedded serial port is channel 2; channel 0 is reserved for a plug-in serial card that the 2080-LC20-20QBB does not have. Documented in `GS10_Integration_Guide.md` §5 and contradicted by the v5.1 LD guide — this doc reconciles them.
2. **Rung 1 reads Output Frequency, but the MSG instance was named `mb_read_status`** — misleading. `0x2103` is the live monitor block, not the status word. Actual status bits live at `0x2101` (a separate FC03 read this doc adds).
3. **ElementCnt mismatch** — comment said 4, struct said 6. `0x2108` is undocumented in the GS10 register map. `ElementCnt := 4` only.

From the deployed `Micro820_v4.1.8_Program.st` (gap audit 2026-05-17):

4. **All four `.Channel := 0`** at lines 251, 259, 267, 275 — must change to `:= 2`.
5. **`read_local_cfg.ElementCnt := 6`** at line 254 — must change to `:= 4`.
6. **Instance named `mb_read_status`** at lines 316, 340, 350 — rename to `mb_read_monitor` (matches what it actually does).
7. **No FC03 read of the real status word** (`0x2101`) — add a second read block to recover the running / at-speed / fault bits.
8. **State machine writes `vfd_cmd_word`** from 6+ places (lines 113, 127, 129, 133, 146, 148, 153, 157, 172, 185). Comment out every one before adding the UDFB — two writers in one scan = last-writer-wins = random behavior.
9. **No `e_stop_ok` global** — CCW LD pin fields don't accept compound boolean expressions, so the UDFB's `EStopOK` pin needs a single tag. Pre-compute it in Prog2 §1.
10. **Poll timer at 500 ms** (line 299) — works, just slow. Tune to 50–100 ms when the bench is steady.
11. **Cmd words 18 / 20 unverified on bench** — three different values appear across sources (10 / 18 / 20 / 34). The bit-math contradicts the "common command words" table on the same page of the integration guide. See Section 18.

Part 3 (Section 20) maps every fix above to the specific edit in the `.st` file so you can apply them line by line.

PART 1 — Modbus polling sequencer (Prog3-Modbus)

1 The Big Picture (Part 1)

The PLC needs to ask the VFD four questions, on a rotation, forever. "What's your output frequency?" "Run forward at 30 Hz." "Frequency setpoint = 30.0 Hz." "Clear any active fault." Each question is one Modbus RTU packet on the RS-485 wire. Each packet is fired by one ladder rung. A counter (`vfd_poll_step`) decides whose turn it is — only one rung sends at a time, because the wire can only carry one packet at a time without collisions.

2 The Real-World Problem

A conveyor sits on the floor with a 2-HP motor wired through a GS10 VFD. The motor's run/stop/direction is controlled from the PLC — not the VFD's keypad. The PLC needs to (a) tell the VFD what to do, (b) read back what the VFD is actually doing, and (c) clear faults when the operator presses the reset button. All of this over one twisted pair of wires running Modbus RTU.

The bench rig is one PLC, one drive, one motor, one ~6-foot RS-485 cable. No production gear. You can break things and restart them. That's the point.

3 RS-485 wiring + GS10 keypad parameter setup (prerequisites)

Both walkthroughs assume the wiring + keypad params are already correct. If you're starting from a brand-new bench, do these first.

RJ45 serial cable to the GS10

```
GS10 RJ45 Serial Comm Port pinout (looking into the jack on the drive):

  ┌──────────────────────────────────────┐
→ │   1   2   3   4   5   6   7   8     │   ← pin numbers
  │   ─   ─   ─   ─   ─   ─   ─   ─     │
  │   ▒   ▒   ●   ●   ●   ▒   ▒   ▒     │
  │           │   │   │                 │
  │           │   │   └── PIN 5 = SG+  (D+, non-inverting)
  │           │   └────── PIN 4 = SG-  (D-, inverting)
  │           └────────── PIN 3 = SGND (signal ground)
  └──────────────────────────────────────┘
                                    [GS10 manual p.10]

Pins 1, 2, 6, 7, 8: NO CONNECT — leave open.
For bench runs < 30 ft, no termination resistor needed.
For runs > 30 ft, 120Ω across D+/D- at the far end.
```

> **GS10 pin order is DIFFERENT from Allen-Bradley PowerFlex**

> YouTube tutorials and Allen-Bradley docs say "pin 4 = D+, pin 5 = D-" because that's how PowerFlex 525 wires it. **The GS10 reverses these:** on the GS10, **pin 5 is D+ (SG+)** and **pin 4 is D- (SG-)**. If you follow the AB convention by mistake you'll have D+/D- swapped — no traffic, no error, just silence on the wire. Verify with a multimeter: ~2.0–2.5V DC idle bias measured pin 5 (+) to pin 4 (-), drive powered, PLC offline.

GS10 keypad parameters (set once, save, power-cycle)

Param	Value	Meaning
P00.20	`3`	Frequency source = RS-485 comms
P00.21	`3`	Run command source = RS-485 comms
P09.00	`1`	Modbus slave address
P09.01	`96`	9600 baud
P09.04	`13`	RTU 8N2 format
P09.07	`2`	Modbus comm-loss action = stop the drive on timeout

✓**You should see:** After saving P09.xx changes and power-cycling the VFD, the keypad shows the steady "STOP" indicator and you can write to `0x2000` from a PC Modbus tester without the drive faulting CE10.

4 Modbus polling — the simple version first

Before the 4-rung sequencer exists, the simplest thing that works is **one** rung firing **one** `MSG_MODBUS` read continuously:

```
    ├─[ ]─always_true─[MSG_MODBUS]─mb_read_one                    │
    │                                                             │
```

One read, every scan. The PLC fires another Modbus packet before the previous one has finished. The drive ignores you. The serial driver buffers up requests and eventually goes out of sync. You see CE10 (comm timeout) on the VFD keypad and `ErrorID = 55` in the live monitor. The simple version reads one register fine for about 200 ms, then everything jams.

> **Why the simple version "works" at all**
> `MSG_MODBUS` 's `IN` pin is rising-edge triggered. Even with `always_true` always high, the block only fires once per scan and waits for `.Done` before re-triggering. So you get one packet every few scans, which "works" for one register. The problem appears the moment you want to read AND write — two messages racing each other.

5 Why the simple version becomes a problem

The conveyor needs four conversations: read monitor data, write the run command, write the frequency setpoint, clear faults. Fire all four from rungs all enabled at the same time and the serial driver tries to send four

packets stacked end-to-end. The Micro 820's embedded serial driver doesn't queue packets — it expects you to wait for one to complete before starting the next. Stack them and you get partial sends, garbled CRCs, and the drive's CE10 timeout fault.

> **The pain made concrete**
> Four people trying to ask the same waiter four different questions at the same time. The waiter can answer one at a time. If everyone shouts, the waiter writes nothing down. You need a queue — and a way to take turns.

6 The better pattern — a step-driven polling sequencer

One integer (`vfd_poll_step`) cycles 1 → 2 → 3 → 4 → 1, advanced by a TON timer in Prog2. Each ladder rung in Prog3 watches the step and fires its `MSG_MODBUS` **only when the step matches its number**. So at any given instant exactly one MSG is active. The wire never sees overlap.

Why this matters: A polling sequencer is a rotating spotlight. Only the rung the spotlight is on can speak. When the spotlight moves on, that rung shuts up and the next rung takes its turn. The TON timer is the clock that moves the spotlight; `vfd_poll_step` is the spotlight's current position. Add a fifth conversation later? Bump the modulo, add Rung 5, done — the existing four don't change.

7 CCW step-by-step — build Prog3-Modbus

Open `MIRA_v5.ccwsln` . Pre-build checklist (most of this lives in Prog2 + the global variable list; if you skipped one, the rung will compile but report `ErrorID = 55` the moment you go online).

Pre-build checklist

1. ☐ CCW project `MIRA_v5` exists; controller is **2080-LC20-20QBB**.
2. ☐ **Embedded Serial Port** → **Properties**: Protocol = Modbus RTU, Mode = Master, Baud = 9600, Parity = None, Stop Bits = 2, Handshake = None.
3. ☐ Variables imported from `VARIABLES_IMPORT.csv` . Includes `vfd_poll_step` , `vfd_poll_active` , four config structs, four data buffers, four `MSG_MODBUS` instances.
4. ☐ Prog2 (ST) loaded with `Micro820_v4.1.8_Program.st` . Prog2 owns the timer that advances `vfd_poll_step` and loads the four config structs each scan.
5. ☐ `MbSrvConf.xml` copied to `Controller\Controller\` folder.
6. ☐ Program execution order: **Prog2 first, Prog3-Modbus second** (Project Organizer → Programs → Properties).

Add the Prog3-Modbus LD program

1. ☐ Project Organizer → right-click **Programs** → **Add** → **New LD: Ladder Diagram**.
2. ☐ Rename it **Prog3-Modbus**. Confirm execution order again.
3. ☐ Double-click to open the ladder editor.

CCW "no match" panic — don't click away

When you drop a contact and type `vfd_poll_step`, the dropdown may briefly show "no match" while indexing. Press Enter anyway — if the variable was imported from CSV, CCW will accept it. Only worry about "no match" if it persists after a build attempt.

Rung 1 — Read VFD Monitor Block

```
 │                                                              │
 ├[EQU]─[XIC]─[MSG_MODBUS]
 │ A = B  vfd_poll_active   mb_read_monitor
 │ A = vfd_poll_step
 │ B = 1
 │                                                              │
```

Reads 4 registers starting at 0x2103 (Output Freq, Current, DC Bus, V_out). Instance is `mb_read_monitor` — v4.1.8 calls it `mb_read_status`, which is misleading because 0x2103 is the monitor block, not the status word.

1. ☐ Drag **EQU** from Toolbox. Source A = `vfd_poll_step`, Source B = `1`.
2. ☐ Drag **XIC** (NO contact). Variable = `vfd_poll_active`.
3. ☐ Drag **MSG_MODBUS** (search "MSG"). Instance name = **`mb_read_monitor`**.
4. ☐ Click each pin to wire variables:
    - `LocalCfg` = `read_local_cfg`
    - `TargetCfg` = `read_target_cfg`
    - `LocalAddr` = `read_data`

Config values Prog2 ST loads each scan (these go into the structs — not the rung):

Field	Value	Why
`read_local_cfg.Channel`	`2`	Embedded RS-485 is channel 2 on Micro 820
`read_local_cfg.Cmd`	`3`	FC03 = Read Holding Registers
`read_local_cfg.ElementCnt`	`4`	4 monitor regs — 0x2103–0x2106. Reading 6 lands in undocumented 0x2108.
`read_target_cfg.Addr`	`8451` (0x2103)	Start of the monitor block *[GS10 manual p.5]*
`read_target_cfg.Node`	`1`	VFD's Modbus slave address (P09.00)

Rung 2 — Write VFD Command

```
 │                                                              │
 ├[EQU]─[XIC]─[MSG_MODBUS]
 │ A = B  vfd_poll_active   mb_write_cmd
 │ A = vfd_poll_step
 │ B = 2
 │                                                              │
```

Writes one register to 0x2000 (Control Command). Value from `vfd_cmd_word`, copied into `write_cmd_data(1)` by Prog2 ST via COP function block.

1. [ ] **EQU**: A = `vfd_poll_step`, B = `2`.
2. [ ] **XIC**: `vfd_poll_active`.
3. [ ] **MSG_MODBUS** instance: `mb_write_cmd`. Pins: `LocalCfg = write_cmd_local_cfg`, `TargetCfg = write_cmd_target_cfg`, `LocalAddr = write_cmd_data`.

Config: `Channel := 2`, `Cmd := 6` (FC06 Write Single Register), `ElementCnt := 1`, `Addr := 8192` (0x2000), `Node := 1`.

> **Why this matters:** Prog2 ST owns `vfd_cmd_word` and copies it into `write_cmd_data(1)` each scan via a COP function block (v4.1.8 line 291). Skip the copy and the write fires with stale data — motor won't start or runs the wrong way.

Rung 3 — Write Frequency Setpoint

```
|                                                                  |
|-[EQU]-[XIC]-[MSG_MODBUS]
|  A = B  vfd_poll_active   mb_write_freq
|  A = vfd_poll_step
|  B = 3
|                                                                  |
```

Writes one register to 0x2001 (Frequency Setpoint, value = Hz × 10).

1. [ ] **EQU**: A = `vfd_poll_step`, B = `3`.
2. [ ] **XIC**: `vfd_poll_active`.
3. [ ] **MSG_MODBUS** instance: `mb_write_freq`. Pins: `LocalCfg = write_freq_local_cfg`, `TargetCfg = write_freq_target_cfg`, `LocalAddr = write_freq_data`.

Config: `Channel := 2`, `Cmd := 6`, `ElementCnt := 1`, `Addr := 8193` (0x2001), `Node := 1`.

> **Frequency is Hz × 10, not Hz**
> To run at 30.0 Hz, load `300` into `vfd_freq_setpoint` — not `30`. Loading `30` sets 3.0 Hz, which is below most motor minimum frequencies and looks like the drive isn't responding. v4.1.8 line 284 already does `conveyor_speed_cmd * 10` — keep it.

Rung 4 — Fault Reset

```
|                                                                  |
|-[EQU]-[XIC]-[MSG_MODBUS]
|  A = B  vfd_poll_active   mb_fault_reset
|  A = vfd_poll_step
|  B = 4
|                                                                  |
```

Writes value `2` to register 0x2002 to clear any active fault.

1. [ ] **EQU**: A = `vfd_poll_step`, B = `4`.
2. [ ] **XIC**: `vfd_poll_active`.
3. [ ] **MSG_MODBUS** instance: `mb_fault_reset`. Pins as above with `fault_reset_*` structs.

Config: `Channel := 2`, `Cmd := 6`, `ElementCnt := 1`, `Addr := 8194` (0x2002), `Node := 1`. Prog2 ST sets `fault_reset_cmd := 2` and copies into `fault_reset_data(1)` via COP (v4.1.8 lines 290, 295).

8 Code walkthrough — one Modbus scan, step by step

Pick a moment when `vfd_poll_step = 2` and `vfd_poll_active = TRUE`. The state machine just decided we want REV+RUN, so `vfd_cmd_word = 20`. Walk one scan:

1. **Input update phase.** Micro 820 reads the physical input table — start button, e-stop, selector. None of those affect Prog3 directly; they affect `vfd_cmd_word` via the state machine in Prog2 (or, after Part 2, via the motor_starter UDFB in Prog4).
2. **Prog2 ST executes.** Loads all four config structs (Channel=2, Cmd, Addr, Node). Loads `write_cmd_data(1) := vfd_cmd_word` = 20 via COP. Advances `vfd_poll_timer`; if `.Q`, increments `vfd_poll_step` and resets.
3. **Prog3 LD executes.** Rung 1's EQU evaluates `vfd_poll_step = 1` → FALSE → rung dead. Rung 2's EQU evaluates `vfd_poll_step = 2` → TRUE. XIC sees `vfd_poll_active` → TRUE. `mb_write_cmd.IN` rises → block builds an FC06 packet at 0x2000 with payload 20 and hands it to the serial driver. Rungs 3 and 4: EQUs are FALSE; their MSGs sit idle.
4. **Output update phase.** No physical output driven from Prog3.
5. **Housekeeping (comms phase).** Serial driver transmits the queued packet. The drive responds ~5–15 ms later (depends on baud + packet length).
6. **Next scan.** Packet is still on the wire. `mb_write_cmd.Done` is FALSE because the drive hasn't replied. Rung 2 still satisfies EQU+XIC but the MSG block won't re-fire — it's internally in "waiting for response" state.
7. **A few scans later.** Drive replies. `mb_write_cmd.Done` goes TRUE for one scan. `vfd_poll_timer` eventually completes; `vfd_poll_step` increments to 3.

Why this matters: The reason rung 2 doesn't re-fire while it's waiting for the drive is internal to `MSG_MODBUS` — the block has its own state machine. EQU+XIC being TRUE is necessary but not sufficient; the block checks "am I currently in a transaction?" If yes, it ignores its `IN` until `.Done` or `.Error` fires. This is what protects the wire from packet stacking.

9 Test Procedure — Part 1 (Modbus only)

1. ☐ Build: Ctrl+Shift+B → **0 errors**. Unknown-variable errors mean the CSV import didn't take — re-run it.
2. ☐ Download via USB (Ethernet sometimes leaves the serial driver out of sync on first download after a structure change).
3. ☐ PLC to Run.
4. ☐ Online → Variable Monitor.

✓You should see: `vfd_poll_step` visibly cycles `1 → 2 → 3 → 4 → 1` at the timer rate. If stuck on one value: Prog2 timer broken, or `vfd_poll_active` stuck FALSE.

✓**You should see:** `mb_read_monitor.ErrorID` = `0` (success) for at least 10 seconds. `255` = MSG never executed (serial port out of sync — re-download). `55` = timeout (check D+/D- swap, P09.04=13).

✓**You should see:** `read_data(1)` > `0` when motor is commanded to run. Value is Hz × 10 (300 ≈ 30.0 Hz).

✓**You should see:** VFD keypad displays the commanded frequency (~30.0 Hz) within 2 seconds of `vfd_cmd_word := 18`.

PART 2 — motor_starter UDFB (run/stop/direction logic)

10 The Big Picture (Part 2)

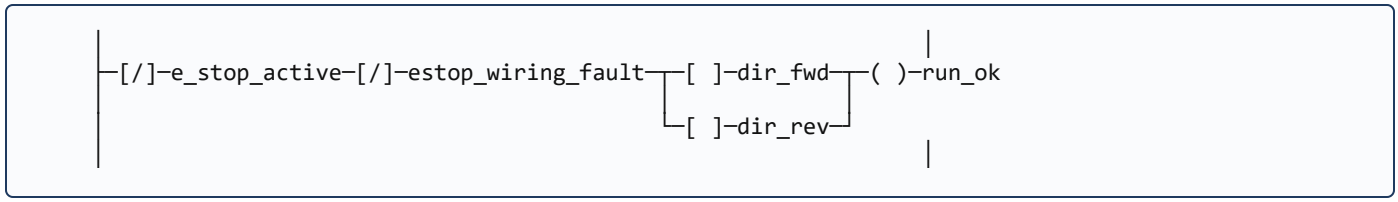
You're going to push a green button. A motor will spin. Whether the motor spins forward or reverse depends on where a 3-position selector switch is pointing when you press the button. Flipping the selector while the motor is running stops it — you have to press the button again to start it back up in the new direction. Pressing the e-stop kills it immediately. Releasing the e-stop alone does not restart the motor; you have to press the green button again.

You're going to build that behavior **once**, inside a sealed box of logic called a User-Defined Function Block (UDFB). The sealed box has labeled terminals on its outside: one for the green button, one for each direction-switch position, one for the e-stop status, one for "enable", and on the output side one terminal that drops out the VFD command word and one that lights the run lamp. Inside the box, the logic is the same every time. Wire one box to one motor. If you ever need a second motor, wire a second copy of the same box. Fixing a bug inside the box fixes both motors at once.

11 Motor logic — the simple version first

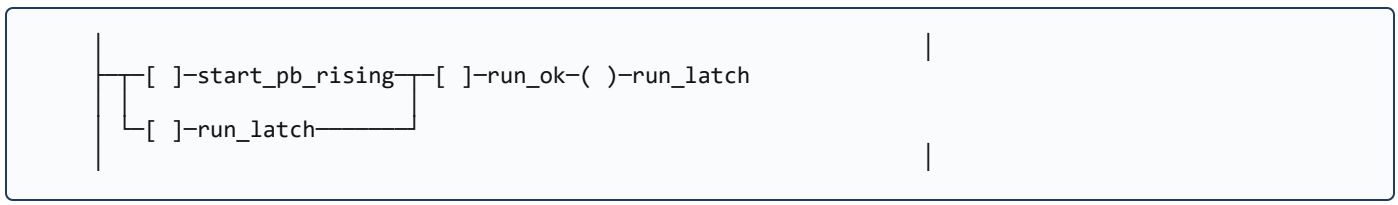
Before the UDFB exists, the simplest correct version of this logic is five flat ladder rungs in Prog3 (or in Prog2 ST). Show them flat first, prove they work, then go look at the pain.

Rung 1 — Drop the seal-in on any stop condition



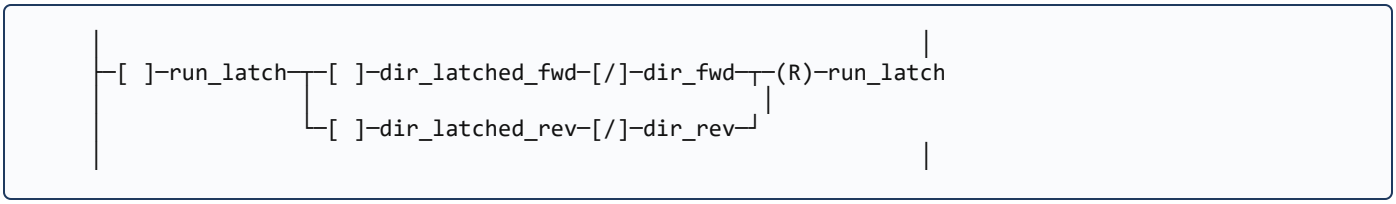
An "OK to run" composite bit. TRUE means e-stop healthy AND a direction is selected.

Rung 2 — Seal-in latch on the green pushbutton



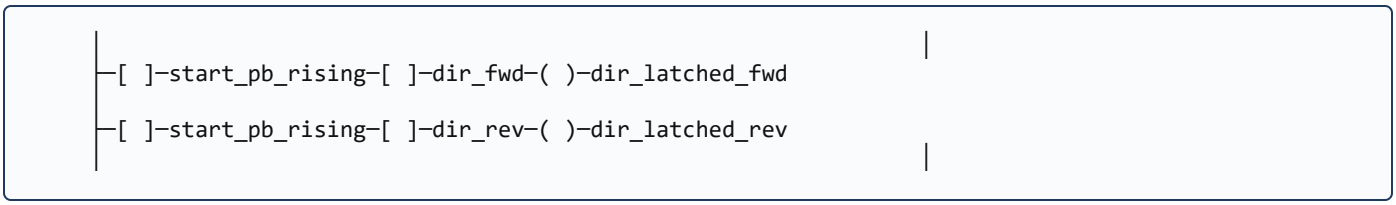
Classic seal-in. `start_pb_rising` *is the rising-edge bit of the green PB;* `run_latch` *holds itself in once started, as long as* `run_ok` *stays TRUE.*

Rung 3 — Stop on direction change while running

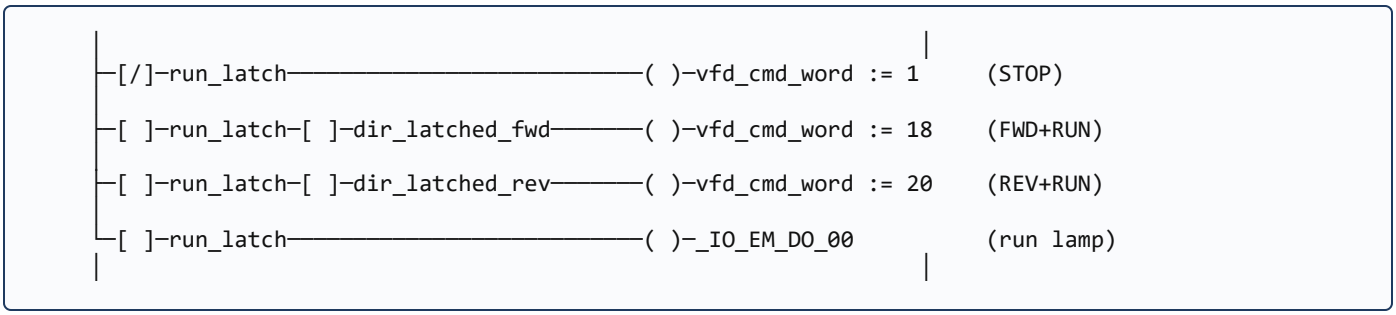


If the direction switch is in a different position than the one captured at start, drop the latch.

Rung 4 — Capture direction at the moment of start



Rung 5 — Assign the VFD command word and the run lamp



> **If you see this, the simple version works**
> Green PB while FWD selector → `vfd_cmd_word = 18` within one scan; VFD keypad shows commanded frequency within ~2 sec; `_IO_EM_DO_00` lights solid; flipping selector to OFF drops `vfd_cmd_word` to `1` within one scan; flipping selector REV mid-run also drops to `1` and requires a fresh PB press.

12 Why the flat rungs become a problem

You wired five rungs. They work. Now imagine a second motor — a discharge cross-belt — needs the same exact behavior: its own green PB, its own selector, its own e-stop, its own run lamp, writing to its own VFD command word over Modbus. To replicate:

- Five rungs become ten rungs.
- Every tag in those new five rungs gets a "_2" suffix or its own new name.
- You find a bug in the seal-in logic. You fix it on motor 1's rungs. Two weeks later, motor 2 has the same bug and you spend an hour wondering why motor 2 doesn't behave like motor 1.
- Ten rungs become fifteen when a third motor arrives. Cognitive load is no longer linear.

> **The pain made concrete (v4.1.8 specifically)**
> The deployed Prog2 ST today has a 5-state machine (`CASE conv_state OF` , line 107) that does roughly the same job, and adding a second conveyor motor would mean copy-pasting the whole `CASE` block with

> renamed locals. The state-machine approach is more powerful than the flat rungs above (timers, fault states, recovery transitions) but it suffers from the same scaling problem when motor-count grows.

13 The better pattern — User-Defined Function Block

A UDFB is a sealed box of logic with labeled input and output terminals. Define it once — name it, declare inputs, declare outputs, declare private locals, write the body. From that point on, every place in the program that needs that behavior drops the block in and wires the terminals. Inside, the logic runs identically every time. The terminals are the contract.

> **Why this matters:** A UDFB is a sealed starter cabinet. You wire field devices to the terminals on the front — start PB, direction switch, e-stop, run lamp, VFD command. Inside is the latching logic, identical in every cabinet. Build one cabinet, install two. Fix a bug in the cabinet design, every cabinet on the floor is now fixed.

The block has **instances**. Each instance is an independent copy of internal storage (`RunLatch` , `DirLatch` , `PrevStartPB`). One motor's instance latches separately from the other's, even though both run the same block code. That's the leverage of the pattern.

14 CCW step-by-step — build the motor_starter UDFB

CCW open with `MIRA_v5` loaded. Create one UDFB (`motor_starter`), call it once from a new Prog4, and remove the conflicting `vfd_cmd_word :=` assignments from Prog2 ST.

Step 1 — Create the UDFB

1. ☐ Project Organizer → right-click **User-Defined Function Blocks** → **Add** → **New Function Block**.
2. ☐ Language: **Structured Text**. (ST is shorter for this case and reads more directly than LD.)
3. ☐ Name: `motor_starter` . OK.
4. ☐ Empty variable list + empty ST body editor opens.

Step 2 — Declare variables (in order — order matters)

Order determines pin order when you drop the block onto a calling rung. Group related pins together.

Section	Name	Type	Initial
VAR_INPUT	`Enable`	`BOOL`	`TRUE`
VAR_INPUT	`StartPB`	`BOOL`	`FALSE`
VAR_INPUT	`DirFwdSw`	`BOOL`	`FALSE`
VAR_INPUT	`DirRevSw`	`BOOL`	`FALSE`
VAR_INPUT	`EStopOK`	`BOOL`	`FALSE`
VAR_OUTPUT	`VFDCmdWord`	`INT`	`1`
VAR_OUTPUT	`RunLamp`	`BOOL`	`FALSE`

Section	Name	Type	Initial
VAR	`RunLatch`	`BOOL`	`FALSE`
VAR	`DirLatch`	`INT`	`0`
VAR	`PrevStartPB`	`BOOL`	`FALSE`
VAR	`StartPBRising`	`BOOL`	`FALSE`
VAR	`DirChangedWhileRunning`	`BOOL`	`FALSE`
VAR	`DropConditions`	`BOOL`	`FALSE`

> **VAR_INPUT default initial — non-obvious gotcha**
> CCW lets you set an initial on a `VAR_INPUT`. That initial is only used if the caller leaves the pin unwired. For `Enable`, defaulting `TRUE` means "if the caller forgets to wire it, the block runs" — what you want for a master gate. For everything else, default `FALSE` so an unwired input never accidentally starts a motor.

Step 3 — Paste the ST body

Click into the ST editor and paste verbatim. ~30 lines.

```
(* === motor_starter — green PB seal-in with direction-lock + e-stop drop === *)

(* Rising-edge on start pushbutton — prevents stuck-PB auto-restart *)
StartPBRising := StartPB AND NOT PrevStartPB;
PrevStartPB := StartPB;

(* Did the operator flip the selector to a different position while running? *)
DirChangedWhileRunning := RunLatch AND (
    (DirLatch = 1 AND NOT DirFwdSw) OR
    (DirLatch = 2 AND NOT DirRevSw)
);

(* Anything that should drop the seal-in latch *)
DropConditions := (NOT Enable)
               OR (NOT EStopOK)
               OR (NOT DirFwdSw AND NOT DirRevSw)   (* selector OFF = stop *)
               OR (DirFwdSw AND DirRevSw)            (* both contacts = wiring fault *)
               OR DirChangedWhileRunning;

(* Seal-in state machine *)
IF DropConditions THEN
    RunLatch := FALSE;
    DirLatch := 0;
ELSIF StartPBRising AND NOT RunLatch THEN
    IF DirFwdSw THEN
        RunLatch := TRUE;
        DirLatch := 1;
    ELSIF DirRevSw THEN
        RunLatch := TRUE;
        DirLatch := 2;
    END_IF;
END_IF;

(* Output assignment *)
IF NOT RunLatch THEN
    VFDCmdWord := 1;            (* STOP *)
ELSIF DirLatch = 1 THEN
```

```
    VFDCmdWord := 18;            (* FWD + RUN — bench-verify, see cheatsheet *)
ELSIF DirLatch = 2 THEN
    VFDCmdWord := 20;            (* REV + RUN — bench-verify, see cheatsheet *)
ELSE
    VFDCmdWord := 1;             (* defensive fallback *)
END_IF;
RunLamp := RunLatch;
```

Step 4 — Build the UDFB

1. ☐ Press **F8**. Look for **0 errors**.
2. ☐ Errors? Check for typos in body vs variable list — `DirFwd` ≠ `DirFwdSw`.

Step 5 — Add the calling rung in a new LD program (Prog4)

1. ☐ Project Organizer → Programs → right-click → **Add** → **New LD: Ladder Diagram** → name it `Prog4-MotorControl`.
2. ☐ Execution order: Prog2 first, Prog3-Modbus second, Prog4-MotorControl third.
3. ☐ Drag a contact onto rung 1, type `enable_motor_ctrl` (new global BOOL, default TRUE).
4. ☐ Toolbox → User-Defined Function Blocks → drag `motor_starter` onto the rung. Instance name: `motor_starter_inst`.
5. ☐ Wire each pin:
    - `Enable` ← wired from the contact
    - `StartPB` ← `_IO_EM_DI_04`
    - `DirFwdSw` ← `dir_fwd`
    - `DirRevSw` ← `dir_rev`
    - `EStopOK` ← `e_stop_ok` (see Step 6)
    - `VFDCmdWord` → `vfd_cmd_word`
    - `RunLamp` → `_IO_EM_DO_00`

> **"EStopOK" can't be a NOT-NOT expression on a ladder pin**
> CCW won't let you type `NOT e_stop_active AND NOT estop_wiring_fault` into the pin field — it expects a single variable. Fix: add one line to Prog2 ST: `e_stop_ok := NOT e_stop_active AND NOT estop_wiring_fault;` and declare `e_stop_ok : BOOL;` as a global. Then wire `EStopOK` ← `e_stop_ok`.

Step 6 — Comment out the conflicting `vfd_cmd_word :=` in Prog2 ST

The UDFB drives `vfd_cmd_word` directly. The old state-machine assignments must be removed so both don't write to the same global within one scan.

1. ☐ Open Prog2.
2. ☐ Find the `CASE conv_state OF` block (v4.1.8 line 107).
3. ☐ Comment out (`(*` … `*)`) every line that assigns to `vfd_cmd_word`. Six sites in v4.1.8: lines 113, 127, 129, 133, 146, 148, 153, 157, 172, 185.
4. ☐ **Leave the rest of the state machine alive.** `conv_state` transitions, `start_timer`, `motor_running`, `item_count`, fault tracking — all keep working. Only the `vfd_cmd_word :=` lines

move to the UDFB.

5. ☐ Save, build (F8), download.

Why this matters: Skip Step 6 and both Prog2 ST and Prog4 LD assign to `vfd_cmd_word` in the same scan, the last writer wins — depends on program execution order. With order Prog2 → Prog3 → Prog4, the UDFB's value wins for one scan, then Prog2 ST overwrites it on the next, and the motor behaves randomly. Comment the old lines now and never wonder later.

15 Code walkthrough — chunk by chunk

Chunk 1 — Edge detection on the start pushbutton

```
StartPBRising := StartPB AND NOT PrevStartPB;
PrevStartPB := StartPB;
```

First line computes "was the button low on the previous scan AND is it high this scan?" — the definition of a rising edge. Second line shifts the current value into the previous-value slot for next scan.

Why this matters: Without rising-edge detection, holding the green PB while the latch is FALSE would re-fire the start every scan as long as the PB stays pressed. With rising-edge, the start only happens once per physical button press. That matters when (a) the operator holds the button — you want one start, not 1000 — and (b) the button welds shut from a stray voltage spike — you don't want the motor to auto-restart every time the seal-in tries to drop.

Chunk 2 — Direction-change-while-running detection

```
DirChangedWhileRunning := RunLatch AND (
    (DirLatch = 1 AND NOT DirFwdSw) OR
    (DirLatch = 2 AND NOT DirRevSw)
);
```

If the seal-in is held AND the captured direction was FWD but the FWD switch contact is no longer closed, the operator must have moved the selector. Same for REV. Either way: the motor is about to be stopped.

Chunk 3 — Drop conditions

```
DropConditions := (NOT Enable)
               OR (NOT EStopOK)
               OR (NOT DirFwdSw AND NOT DirRevSw)
               OR (DirFwdSw AND DirRevSw)
               OR DirChangedWhileRunning;
```

Five reasons to drop, OR'd. Each on its own line so you can read them top-to-bottom: master disable; e-stop unhealthy; selector OFF; selector wiring fault; operator moved the selector mid-run.

Why this matters: Computing `DropConditions` as a single bool before the IF means the latch logic stays short. Inlining all five into the IF directly would make a tangle of nested booleans that's hard to read and hard to extend.

Chunk 4 — The seal-in state machine

```
IF DropConditions THEN
    RunLatch := FALSE;
    DirLatch := 0;
ELSIF StartPBRising AND NOT RunLatch THEN
    IF DirFwdSw THEN
        RunLatch := TRUE;
        DirLatch := 1;
    ELSIF DirRevSw THEN
        RunLatch := TRUE;
        DirLatch := 2;
    END_IF;
END_IF;
```

Three branches. **If** any drop condition is true → kill the latch + clear direction. **Else if** the button just went high AND we aren't already running → try to latch in whichever direction the selector points; if neither (selector OFF), don't latch. **Otherwise**: do nothing.

> **Order of branches matters**
>
> Drop comes *before* start. If both happened in the same scan (operator presses the button at the exact moment the e-stop is released), the drop wins and the start request is ignored. Safer order — you don't want a marginal scan timing to accidentally start a motor.

Chunk 5 — Output assignment

```
IF NOT RunLatch THEN
    VFDCmdWord := 1;
ELSIF DirLatch = 1 THEN
    VFDCmdWord := 18;
ELSIF DirLatch = 2 THEN
    VFDCmdWord := 20;
ELSE
    VFDCmdWord := 1;
END_IF;
RunLamp := RunLatch;
```

Latch not held → STOP. Latch held in FWD → FWD+RUN. Latch held in REV → REV+RUN. Defensive fallback to STOP if the latch is somehow TRUE but the captured direction is 0 (shouldn't happen, but the ELSE is cheap insurance against a future code change). Run lamp mirrors the latch.

16 Scan cycle — UDFB through Modbus, end to end

Five-phase loop, ~10 ms typical. Walk the realistic scenario: operator presses Green PB while selector is in FWD.

Scan	What happens inside the UDFB instance	What appears on the wire / lamp
N	`StartPB` = TRUE, `PrevStartPB` = FALSE → `StartPBRising` = TRUE. `DirFwdSw` = TRUE, `DirRevSw` = FALSE → no drop. ELSIF fires: `RunLatch := TRUE`, `DirLatch := 1`. Output:	`vfd_cmd_word` global = 18; `_IO_EM_DO_00` goes high at end-of-scan.

Scan	What happens inside the UDFB instance	What appears on the wire / lamp
	`VFDCmdWord := 18`, `RunLamp := TRUE`. `PrevStartPB := TRUE`.	
N+1	`StartPB` still TRUE, `PrevStartPB` = TRUE → `StartPBRising` = FALSE. No drop. ELSIF doesn't fire (latch already held). Outputs unchanged.	`vfd_cmd_word` still 18. Prog3-Modbus's sequencer eventually ships an FC06 write of 18 to register `0x2000` — drive starts ramping the motor up. Run lamp solid.
N+~50	Operator releases the button. `StartPB` = FALSE → `StartPBRising` = FALSE. No drop. Latch holds.	Motor still running. Run lamp still solid.
N+~2000	Operator flips selector to REV without stopping first. `DirFwdSw` = FALSE, `DirRevSw` = TRUE. `DirLatch` still = 1 → `DirChangedWhileRunning` = TRUE → `DropConditions` = TRUE. IF fires: `RunLatch := FALSE`, `DirLatch := 0`. Output: `VFDCmdWord := 1`, `RunLamp := FALSE`.	`vfd_cmd_word` drops to 1; drive begins coast-to-stop (or decel-to-stop, depending on GS10 P09.xx settings). Run lamp dark within one scan.
N+~2050	Operator presses green PB again, selector now in REV. `StartPBRising` = TRUE. `DirRevSw` = TRUE → ELSIF sets `RunLatch := TRUE`, `DirLatch := 2`. Output: `VFDCmdWord := 20`.	Motor begins ramping in reverse.

Why this matters: The actual Modbus packet transmit happens in the housekeeping phase at the end of each scan, not at the moment the UDFB writes `vfd_cmd_word`. So you'll see the global update instantly in CCW's variable monitor, but the drive sees the change one Modbus polling cycle later (~500 ms in the deployed sequencer cadence). That's why the keypad takes a beat to display the commanded frequency after you press the button.

17 Test Procedure — Part 2 (UDFB + integration)

1. ☐ Build (F8). 0 errors. (If errors mention `e_stop_ok` undeclared, you skipped Step 5's note — add the global + the Prog2 ST line that computes it.)
2. ☐ Download via USB.
3. ☐ PLC to Run.
4. ☐ Online → Variable Monitor. Pin `vfd_cmd_word`, `motor_starter_inst.RunLatch`, `motor_starter_inst.DirLatch`.

Test 1 — Green PB while selector in OFF → no start

✓**You should see:** Press + release green PB, selector centered. `RunLatch` stays FALSE. `vfd_cmd_word` stays 1. `_IO_EM_DO_00` dark.

Test 2 — Green PB while selector in FWD → forward start

✓**You should see:** Flip to FWD. Press PB. `RunLatch` goes TRUE. `DirLatch` = 1. `vfd_cmd_word` = 18. `_IO_EM_DO_00` solid green. Within ~2 sec VFD keypad shows commanded frequency.

Test 3 — Selector to OFF while running → stop

✓**You should see:** With motor running forward, flip selector to OFF. `RunLatch` drops to FALSE within one scan. `vfd_cmd_word` drops to 1. Motor coasts to stop. Run lamp dark.

Test 4 — Direction-change-while-running drops latch

✓**You should see:** Restart in FWD. While running, flip selector directly from FWD to REV. `RunLatch` drops to FALSE. `vfd_cmd_word` drops to 1. Motor stops. Operator must press PB again to start in REV.

Test 5 — Verify REV actually spins motor reverse (CRITICAL — bench-verify)

✓**You should see:** From stopped, selector in REV, press PB. `vfd_cmd_word` = 20. Within ~2 sec motor spins reverse. **If motor spins forward, the deployed REV value (20) is wrong on your firmware — try 34 (see Section 18).**

Test 6 — E-stop drops everything

✓**You should see:** Motor running, press e-stop. `e_stop_active` TRUE, `e_stop_ok` FALSE, `RunLatch` drops within one scan. Motor stops. Contactor (`_IO_EM_DO_02`) drops open via separate e-stop logic.

Test 7 — E-stop release alone does NOT restart

✓**You should see:** Release e-stop. `RunLatch` stays FALSE — motor does NOT auto-restart. Press PB to restart.

Test 8 — Stuck-PB protection

✓**You should see:** Force-set `_IO_EM_DI_04` = TRUE in CCW monitor. Motor starts once. Flip to OFF (drops latch). Motor stays stopped even though `StartPB` still TRUE — rising-edge prevents auto-restart. Release force.

Pass / Fail summary (combined Modbus + UDFB)

Test	Pass	Fail	Notes
Project builds with 0 errors	☐	☐	
Modbus: vfd_poll_step cycles 1→4	☐	☐	
Modbus: mb_read_monitor.ErrorID = 0 sustained	☐	☐	
Modbus: read_data(1) non-zero when running	☐	☐	
UDFB Test 1 — PB in OFF doesn't start	☐	☐	
UDFB Test 2 — PB in FWD starts forward	☐	☐	
UDFB Test 3 — Selector OFF stops	☐	☐	
UDFB Test 4 — Direction change drops latch	☐	☐	
UDFB Test 5 — REV actually reverses (BENCH-VERIFY)	☐	☐	If fails: try 34 instead of 20
UDFB Test 6 — E-stop drops everything	☐	☐	
UDFB Test 7 — E-stop release alone doesn't restart	☐	☐	
UDFB Test 8 — Stuck PB protected by rising-edge	☐	☐	

Test	Pass	Fail	Notes
End-to-end — fault reset clears CE10	☐	☐	

18 Open question — bench-verify before relying on production

Command word bit math has a known contradiction

Three different values for "REV + RUN" appear across the existing documentation:

- **20** (`0x0014`) — `plc/LD_BUILD_GUIDE_v5.md` Rung 2; `GS10_Integration_Guide.md` §2 "Common Command Words" table; deployed Prog2 ST line 129/148.
- **18** (`0x0012`) — same sources, but labeled as FWD+RUN there.
- **34** (`0x0022`) — `~/.claude/skills/plc-instruction-guide/references/style-vocabulary.md` bug example asserting REV+RUN with explicit bit math "bit 5 + bit 1 = 32 + 2 = 34".

Per `GS10_Integration_Guide.md` §2's bit-field table: RUN = bit 1 (dec 2), FWD = bit 3 (dec 8), REV = bit 4 (dec 16). That arithmetic gives FWD+RUN = 10 and REV+RUN = 18 — neither matches the "Common Command Words" table on the same page (which lists FWD+RUN = 18 and REV+RUN = 20). The skill's 34 corresponds to yet another bit convention (bit 5 for REV).

What to do: bench-test all three values when you sit down at the rig. Whichever value *actually* spins the motor in reverse on your specific GS10 firmware is the correct one. Update the cheatsheet + the integration guide + this doc with the bench-verified value and a `[verified on bench YYYY-MM-DD]` citation. Delete the wrong values from the cheatsheet and the v5.1 build guide so they don't propagate further.

19 Unified troubleshooting

Symptom	Likely cause	What to check	Fix
`mb_*.ErrorID = 255` on every rung	Serial driver not in sync (channel wrong, or partial download)	Serial Port → Diagnose. Status should say "in sync".	Confirm `Channel := 2` in all four cfg structs. Re-download over USB.
`mb_*.ErrorID = 55` (timeout)	Wiring (D+/D- swap), baud mismatch, slave node wrong, or P09.04 wrong	Keypad: P09.00=1, P09.01=96, P09.04=13. Multimeter D+/D-: 2.0–2.5V DC idle bias.	Fix wiring or VFD params. Power-cycle the VFD after changing P09.xx.
Build fails: "unknown identifier `mb_read_monitor`"	Renamed instance but didn't update Prog2 ST references	Search Prog2 for `mb_read_status`.	Find/Replace `mb_read_status` → `mb_read_monitor`. Rebuild.

Symptom	Likely cause	What to check	Fix
Frequency write succeeds but motor runs wrong speed	Forgot the × 10 scaling	`vfd_freq_setpoint` value vs VFD displayed Hz	Multiply desired Hz by 10. 30 Hz → 300.
Direction command runs wrong way	Bit-math contradiction (see §18)	Try 18 / 20 / 34 on bench	Document verified value. Update cheatsheet.
Motor runs once then CE10 / F30	PLC scan stretched and poll timer collided with previous MSG completion	Watch `uptime_seconds` for scan-time spikes	Increase `vfd_poll_timer.PT` from 500ms to 1000ms.
Motor never starts; `RunLatch` stays FALSE no matter what	One of the drop conditions is permanently TRUE	Variable monitor: `Enable`, `EStopOK`, `DirFwdSw`, `DirRevSw`	Most common: forgot `enable_motor_ctrl := TRUE`.
Motor starts but immediately stops on same scan	Forgot Step 6 — Prog2 ST still assigns `vfd_cmd_word := 1` in IDLE	Search Prog2 for `vfd_cmd_word :=` — should be 0 hits after the refactor	Comment out all sites per Part 3 line list.
Direction change DOESN'T drop latch — motor reverses live	`DirLatch` not captured, or chunk 2 typo	Variable monitor: `motor_starter_inst.DirLatch` while running — should be 1 or 2	If DirLatch is 0 while RunLatch is TRUE, chunk 4 ELSIF didn't fire — recheck rising-edge.
Live monitor shows UDFB body but values frozen	Viewing **definition**, not instance	Project Organizer → Programs → Prog4-MotorControl → expand. Find `motor_starter_inst` in local variables.	Monitor the instance, not the definition.
After rebuild, `RunLatch` = TRUE on first scan even though motor wasn't running	CCW retained instance memory across downloads	Compare before/after download	Project → Clear Memory Retention before re-download. Or power-cycle PLC.
Build fails: "undefined identifier `e_stop_ok`"	Added the global to Prog2 ST body but didn't declare in global variable list	Global Variables view → search `e_stop_ok`	Add `e_stop_ok : BOOL;` to globals. Rebuild.
Fault won't clear even with rung 4 firing	Drive in F30.x latched (comm-loss latch)	Keypad: STOP/RESET button. If still latched, power-cycle.	Hardware reset; software is only enough for non-latched faults.

PART 3 — Apply to v4.1.9: line-by-line cleanup of the deployed program

20 Cutting v4.1.9 from `Micro820 v4.1.8 Program.st`

The deployed program has every fix from Parts 1 and 2 still *unapplied*. The TODO comment block at the top of the `.st` file lists each gap. This section maps each gap to the exact edit. Apply in order — each step verifies before the next.

Fix 1 — Four `Channel := 0` assignments → `Channel := 2`

Where: v4.1.8 lines 251, 259, 267, 275.

```
(* BEFORE *)
  read_local_cfg.Channel := 0;
  write_cmd_local_cfg.Channel := 0;
  write_freq_local_cfg.Channel := 0;
  fault_reset_local_cfg.Channel := 0;

(* AFTER *)
  read_local_cfg.Channel := 2;
  write_cmd_local_cfg.Channel := 2;
  write_freq_local_cfg.Channel := 2;
  fault_reset_local_cfg.Channel := 2;
```

✓**You should see:** After download: `mb_read_monitor.ErrorID` changes from 255 (out-of-sync) to either 0 (success) or 55 (timeout). 255 specifically means the channel was wrong — eliminating it confirms this fix took.

Fix 2 — `read_local_cfg.ElementCnt := 6` → `:= 4`

Where: v4.1.8 line 254.

```
(* BEFORE *)
  read_local_cfg.ElementCnt := 6; (* read 6 regs: freq,curr,dcbus,volts,torque,rpm *)

(* AFTER *)
  read_local_cfg.ElementCnt := 4; (* read 4 documented regs: 0x2103..0x2106 *)
```

The comment claimed torque + rpm but those live at 0x2108 / 0x2109, undocumented in the GS10 register map at §2. Reading them returns garbage or triggers Modbus exception 02.

Fix 3 — Rename `mb_read_status` → `mb_read_monitor`

Where: v4.1.8 lines 316, 340, 350 (three references).

In CCW: Project Organizer → Prog3-Modbus → expand → right-click `mb_read_status` → **Rename** → `mb_read_monitor`. CCW updates the LD rung automatically. Then update Prog2 ST:

```
(* BEFORE *)
  mb_read_status(IN := (vfd_poll_step = 1) AND vfd_poll_active, ...);
  IF mb_read_status.Q THEN ...
  IF mb_read_status.Error THEN ...

(* AFTER *)
  mb_read_monitor(IN := (vfd_poll_step = 1) AND vfd_poll_active, ...);
  IF mb_read_monitor.Q THEN ...
  IF mb_read_monitor.Error THEN ...
```

✓**You should see:** Build with 0 errors after the rename — confirms all references updated.

Fix 4 — Add second FC03 read for the actual status word at 0x2101

Where: NEW. Insert below the existing read config block (around line 256 after the renames).

```
(* --- Configure Modbus READ STATUS WORD parameters --- *)
  read_status_local_cfg.Channel := 2;
  read_status_local_cfg.TriggerType := 0;
  read_status_local_cfg.Cmd := 3;        (* FC03 = read holding registers *)
  read_status_local_cfg.ElementCnt := 1; (* one register: status word *)
  read_status_target_cfg.Addr := 8449;   (* 0x2101 = GS10 operation status word *)
  read_status_target_cfg.Node := 1;
```

Add a 5th MSG block instance + 5th LD rung in Prog3-Modbus (EQU vfd_poll_step = 5, MSG instance `mb_read_status_word`, wired to the new structs + a new `read_status_data(1)` INT). Bump the poll-step modulo in Prog2 ST from 4 to 5:

```
(* BEFORE *)
  IF vfd_poll_step > 4 THEN
    vfd_poll_step := 1;
  END_IF;

(* AFTER *)
  IF vfd_poll_step > 5 THEN
    vfd_poll_step := 1;
  END_IF;
```

Then decode the status bits — add to the CASE block at line 338:

```
      5: (* READ STATUS WORD results *)
        IF mb_read_status_word.Q THEN
          vfd_status_word := read_status_data(1);
          vfd_running     := (vfd_status_word AND 16#0002) <> 0;  (* bit 1 *)
          vfd_at_speed    := (vfd_status_word AND 16#0008) <> 0;  (* bit 3 *)
          vfd_fault_active:= (vfd_status_word AND 16#0080) <> 0;  (* bit 7 *)
          vfd_msg_done := TRUE;
          vfd_poll_active := FALSE;
        END_IF;
        IF mb_read_status_word.Error THEN
          vfd_comm_err := TRUE;
          vfd_msg_done := TRUE;
          vfd_poll_active := FALSE;
        END_IF;
```

> **Status word bit positions — bench-verify per drive firmware**
> GS10 manual p.5 lists the status word bits; some firmware revisions reorder them. After this fix, run the motor in FWD and watch `vfd_status_word` in CCW monitor — bit 1 should toggle TRUE when the drive is running. If a different bit toggles, adjust the bit-mask constants.

Fix 5 — Comment out every `vfd_cmd_word :=` assignment in Prog2 ST

Where: v4.1.8 lines 113, 127, 129, 133, 146, 148, 153, 157, 172, 185. **Ten sites total.**

```
(* Example: line 113 *)
(* BEFORE *)
    0: (* IDLE -- waiting for RUN command *)
      motor_running := FALSE;
      conveyor_running := FALSE;
      motor_speed := 0;
      vfd_cmd_word := 1;  (* GS10 stop = 0x0001 *)

(* AFTER *)
    0: (* IDLE -- waiting for RUN command *)
      motor_running := FALSE;
      conveyor_running := FALSE;
      motor_speed := 0;
      (* vfd_cmd_word now owned by motor_starter UDFB in Prog4 *)
      (* vfd_cmd_word := 1; -- removed v4.1.9 *)
```

Repeat for all ten sites. Leave the state machine alive — `conv_state` transitions, `motor_running`, `start_timer`, `fault_alarm` tracking all continue to work.

> ✓**You should see:** Search Prog2 for `vfd_cmd_word :=` after the edits — should return **0 hits**.

Fix 6 — Add `e_stop_ok` global + computation in Prog2 ST §1

Where: declare in CCW Controller Variables; compute in Prog2 ST around line 73 (after the e-stop supervision IF block).

```
(* AFTER the existing IF (_IO_EM_DI_02 XOR _IO_EM_DI_03) THEN ... END_IF; *)
  e_stop_ok := NOT e_stop_active AND NOT estop_wiring_fault;
```

In CCW: Global Variables → add row → Name = `e_stop_ok`, Type = `BOOL`, Initial = `FALSE`.

Fix 7 — Create `motor_starter` UDFB + Prog4-MotorControl + wire the instance

Where: NEW Prog4 program + NEW UDFB. Follow Section 14 Steps 1–5 of Part 2 verbatim.

> ✓**You should see:** After build, the global `vfd_cmd_word` updates within one scan of pressing the green PB. CCW Variable Monitor: pin `vfd_cmd_word` + `motor_starter_inst.RunLatch`.

Fix 8 — Bench-verify FWD+RUN / REV+RUN values

Section 18 procedure. Update the cheatsheet, the integration guide, and the UDFB output assignment with the bench-verified values + a `[verified on bench YYYY-MM-DD]` citation.

Fix 9 — (Optional) Tune poll timer from 500 ms to 100 ms

Where: v4.1.8 line 299. Only after Fixes 1–8 are confirmed working on the bench.

```
(* BEFORE *)
  vfd_poll_timer(IN := NOT vfd_poll_active, PT := T#500ms);

(* AFTER *)
  vfd_poll_timer(IN := NOT vfd_poll_active, PT := T#100ms);
```

✓**You should see:** After tuning: drive responds to command changes within ~200 ms instead of ~1 sec. If `vfd_comm_err` starts ticking up, scan time can't keep up — revert to 500 ms.

Final v4.1.9 changelog block (add to top of `.st`)

```
(*  v4.1.9 - Applied every fix flagged in                          *)
(*           docs/instructions/Conv_Simple_Complete.pdf §20:        *)
(*           - Channel 0 -> 2 (4 places)                           *)
(*           - ElementCnt 6 -> 4 on monitor read                   *)
(*           - Renamed mb_read_status -> mb_read_monitor           *)
(*           - Added FC03 status word read at 0x2101               *)
(*           - Removed 10 vfd_cmd_word := assignments from Prog2   *)
(*           - Added e_stop_ok global + computation                *)
(*           - Built motor_starter UDFB + Prog4-MotorControl       *)
(*           - Bench-verified FWD+RUN=NN, REV+RUN=NN (YYYY-MM-DD)  *)
(*           - Tuned poll timer to 100ms (was 500ms)               *)
*)
```

21 Mental model — the whole rig in one breath

Why this matters: The Micro 820 is a clerk with three notebooks open: Prog2 ST (the brain — tracks state, decides what should happen), Prog3-Modbus LD (the radio operator — takes turns talking on the serial wire, one packet at a time), and Prog4-MotorControl LD (a sealed starter cabinet — wires the green button + selector + e-stop into a clean run-or-don't decision that overwrites the global the radio ships). Every ~10 ms the clerk reads physical inputs, runs each notebook in order, writes physical outputs, then has the radio actually transmit. The contradiction in the bit-math tables and the historical "channel := 0" drift between docs are the lessons in why every value in this whole stack needs a citation — when you can't trace a number to a manual page or a bench-verified run, it's a guess waiting to bite the next person.

22 What you should understand now

- **Why `Channel := 2`** and not 0 for the Micro 820 embedded serial port (channel 0 = plug-in serial card that the 2080-LC20-20QBB doesn't have).
- **Why every MSG_MODBUS instance is independent.** Each has its own internal state — sharing one across rungs collides on the wire.
- **The difference between status word (0x2101) and monitor block (0x2103+).** v5.1 conflated them; v4.1.9 separates them.
- **Why frequency is Hz × 10.** GS10's register is encoded that way.

- **Why rising-edge is the right trigger for "start".** Level-triggered allows stuck buttons to auto-restart — unsafe.
- **Why drop conditions are computed before the latch IF.** One named bool keeps the body readable and makes a new drop reason a one-line addition.
- **Why each UDFB instance has its own RunLatch.** Per-instance locals are the leverage of the pattern.
- **Why EStopOK is an input, not a hardcoded global inside the UDFB.** A second instance could be gated by a different safety chain.
- **Why VFDCmdWord is an INT, not a BOOL.** GS10's command word is a bit-field integer, not a single bit.
- **Why Prog2 ST's `vfd_cmd_word :=` lines had to be commented out.** Two writers in one scan = unpredictable.
- **Why the live monitor shows different values depending on instance vs definition.** Definition is the code template; instance is the runtime data.
- **Why every value in this doc cites a source.** The 18/20/34 contradiction is what happens when a number gets copy-pasted without one.

23 Next Step

In order of leverage:

1. **Bench-verify the FWD+RUN / REV+RUN command words** and update every doc + the .st with the verified values. This unblocks every other downstream task.
2. **`Conv_Simple_StatusWord_Decode.html`** — once Fix 4 above is shipped, write a beginner walkthrough on what each bit of `vfd_status_word` means + how to consume `vfd_running` / `vfd_at_speed` / `vfd_fault_active` from the UDFB or HMI.
3. **`Conv_Simple_UDFB_DualMotor.html`** — instantiate `motor_starter` twice for a second conveyor motor on its own VFD (Modbus slave 2). Demonstrates the UDFB payoff: one definition, two instances, two independent motors, no copy-paste.
4. **`Conv_Simple_UDFB_VFDFeedback.html`** — extend `motor_starter` with a `MinFreqOK` input that gates the start until the drive reports a stable DC bus. Shows how the UDFB's contract grows as you wire in feedback.

ⓘ Tear-off appendix — combined back-of-doc reference

v4.1.9 fix checklist (apply in order)

1. ☐ Channel 0 → 2 (4 sites: lines 251, 259, 267, 275)
2. ☐ ElementCnt 6 → 4 (line 254)
3. ☐ Rename `mb_read_status` → `mb_read_monitor` (3 sites in .st + LD rung)
4. ☐ Add FC03 status read at 0x2101 + 5th rung + status decode
5. ☐ Comment out 10× `vfd_cmd_word :=` in Prog2 (lines 113, 127, 129, 133, 146, 148, 153, 157, 172, 185)

6. [ ] Add `e_stop_ok` global + computation in Prog2 §1
7. [ ] Build `motor_starter` UDFB + Prog4-MotorControl + wire instance
8. [ ] Bench-verify FWD+RUN / REV+RUN command words
9. [ ] (Optional) tune poll timer 500ms → 100ms (line 299)
10. [ ] Update top-of-file changelog comment to v4.1.9

Critical values (must be right)

Channel (embedded RS-485)	`2`
Baud / format	`9600 8N2 RTU`
Drive Modbus node	`1`
RJ45 pin 5 = SG+; pin 4 = SG-; pin 3 = SGND	—
FC03 monitor block start	`0x2103`, ElementCnt = 4
FC03 status word	`0x2101`, ElementCnt = 1
FC06 Control Command	`0x2000`
FC06 Frequency Setpoint	`0x2001`, value = Hz × 10
FC06 Fault Reset	`0x2002`, data = `2`
Cmd: STOP	`1`
Cmd: FWD + RUN (bench-verify)	`18`
Cmd: REV + RUN (bench-verify)	`20`

UDFB drop conditions (anything that stops the motor)

- `NOT Enable`
- `NOT EStopOK`
- Selector OFF (`NOT DirFwdSw AND NOT DirRevSw`)
- Selector wiring fault (`DirFwdSw AND DirRevSw`)
- Direction change while running (`RunLatch AND captured-dir ≠ live-dir`)